

Database Management Systems

Shamim Ahmad



Relational Database Design

Functional Dependencies (Cont.)

- A functional dependency is **trivial** if it is satisfied by **all instances of a relation**

★ *E.g.*

✓ *customer-name, loan-number $\rightarrow$ customer-name*

✓ *customer-name $\rightarrow$ customer-name*

★ In general, $\alpha \rightarrow \beta$ is **trivial** if $\beta \subseteq \alpha$

Functional Dependencies (Cont.)

Some functional dependencies are said to be **trivial** because they are satisfied by **all relations**.

For example, $A \rightarrow A$ is satisfied by all relations involving attribute A.

Closure of a Set of Functional Dependencies

- Given a set F set of functional dependencies, there are certain **other functional dependencies** that **are logically implied** by F .
 - ★ E.g. If $A \rightarrow B$ and $B \rightarrow C$, then we can infer that $A \rightarrow C$
- The set of all functional dependencies logically implied by F is the **closure** of F .
- We denote the **closure** of F by **F^+** .

Functional Dependencies (Cont.)

$A \rightarrow B$

$A \rightarrow C$

$CG \rightarrow H$

$CG \rightarrow I$

$B \rightarrow H$

The functional dependency

$A \rightarrow H$

Closure of a Set of Functional Dependencies

■ We can find all of F^+ by applying Armstrong's Axioms:

- ★ if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$ (reflexivity)
- ★ if $\alpha \rightarrow \beta$, then $\gamma \alpha \rightarrow \gamma \beta$ (augmentation)
- ★ if $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$ (transitivity)

■ These rules are

- ★ **sound** (generate only functional dependencies that **actually hold**) and
- ★ **complete** (generate **all functional** dependencies that hold).

Closure of Functional Dependencies (Cont.)

■ We can further simplify manual computation of F^+ by using the following additional rules.

★ If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta \gamma$ holds
(**union**)

★ If $\alpha \rightarrow \beta \gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds
(**decomposition**)

★ If $\alpha \rightarrow \beta$ holds and $\gamma \beta \rightarrow \delta$ holds, then $\alpha \gamma \rightarrow \delta$ holds
(**pseudotransitivity**)

The above rules can be inferred from **Armstrong's axioms**.

Example

■ $R = (A, B, C, G, H, I)$

$$F = \left\{ \begin{array}{l} A \rightarrow B \\ A \rightarrow C \\ CG \rightarrow H \\ CG \rightarrow I \\ B \rightarrow H \end{array} \right\}$$

Example

some members of F^+

★ $CG \rightarrow HI$

✓ from $CG \rightarrow H$ and $CG \rightarrow I$: “union rule” can be inferred from

— definition of functional dependencies, **Or**

—

— Augmentation of $CG \rightarrow I$

» to infer $CG \rightarrow CGI$ {that is, $CGCG \rightarrow CGI$ }

— Augmentation of $CG \rightarrow H$

» to infer $CGI \rightarrow HI$ {that is, $CGI \rightarrow CGI$ }

— And transitivity

» $CG \rightarrow HI$

■ $R = (A, B, C, G, H, I)$
 $F = \{$
 $A \rightarrow B$
 $A \rightarrow C$
 $CG \rightarrow H$
 $CG \rightarrow I$
 $B \rightarrow H\}$

Procedure for Computing F^+

■ To compute the closure of a set of functional dependencies F :

$F^+ = F$

repeat

for each functional dependency f in F^+

 apply **reflexivity and augmentation** rules on f

 add the resulting functional dependencies to F^+

for each pair of functional dependencies f_1 and f_2 in F^+

if f_1 and f_2 can be combined using **transitivity**

then add the resulting functional dependency to F^+

until F^+ does not change any further

NOTE: *We will see an alternative procedure for this task later*

Closure of Attribute Sets

- Given a set of attributes α , define the **closure** of α **under** F (denoted by α^+) as the **set of attributes** that are functionally **determined by α under F** :

$$\alpha \rightarrow \beta \text{ is in } F^+ \implies \beta \subseteq \alpha^+$$

- Algorithm to compute α^+ , the closure of α under F
result := α ;
 while (changes to *result*) **do**
 for each $f_1 \rightarrow f_2$ **in** F **do**
 begin
 if $f_1 \subseteq \text{result}$ **then** *result* := *result* $\cup$ f_2
 end

Example of Attribute Set Closure

■ $R = (A, B, C, G, H, I)$

■ $F = \{A \rightarrow B$
 $A \rightarrow C$
 $CG \rightarrow H$
 $CG \rightarrow I$
 $B \rightarrow H\}$

Example of Attribute Set Closure

■ $(AG)^+$

1. $result = AG$

2. $result = ABCG$ $(A \rightarrow C \text{ and } A \rightarrow B)$

3. $result = ABCGH$ $(CG \rightarrow H \text{ and } CG \subseteq AGBC)$

4. $result = ABCGHI$ $(CG \rightarrow I \text{ and } CG \subseteq AGBCH)$

■ $R = (A, B, C, G, H, I)$

■ $F = \{A \rightarrow B$
 $A \rightarrow C$
 $CG \rightarrow H$
 $CG \rightarrow I$
 $B \rightarrow H\}$

Closure of Functional Dependencies (Cont.)

- Given the following FDs
- $A \rightarrow BC$, $CD \rightarrow E$, $E \rightarrow C$
- $D \rightarrow AEH$, $ABH \rightarrow BD$, $DH \rightarrow BCA$
- Find out, whether $BCD \rightarrow H$ or not from the following Functional Dependencies?
- **Solution** : Compute the closure of
- $BCD = BCD^+$
- $BCD^+ = BCD$
 $= ABCDEH$ {as $D \rightarrow AEH$ }
- Hence $BCD \rightarrow H$ is true as the closure contains H.

Example of Attribute Set Closure

- Suppose we are given relation R with attributes A,B,C,D and

FDs $A \rightarrow BC$ $B \rightarrow CD$

- $(B^+) = \{BCD\}$
- $(C^+) = \{C\}$
- $(D^+) = \{D\}$
- $(AB^+) = \{ABCD\}$
- $(AC^+) = \{ABCD\}$
- $(AD^+) = \{ABCD\}$
- $(BC^+) = \{BCD\}$
- $(BD^+) = \{BCD\}$
- $(CD^+) = \{CD\}$

Example of Attribute Set Closure

■ relation schema $R = \{A, B, C, D, E\}$.

★ $A \rightarrow BC$

★ $CD \rightarrow E$

★ $B \rightarrow D$

★ $E \rightarrow A$

Example of Attribute Set Closure

- $(A)^+ = \{ABCDE\}$
- $(AB)^+ = \{ABCDE\}$
- $(AC)^+ = \{ABCDE\}$
- $(BC)^+ = \{ABCDE\}$
- $(BD)^+ = \{BD\}$
- $(BE)^+ = \{ABCDE\}$
- $(CD)^+ = \{ABCDE\}$
- $(CE)^+ = \{ABCDE\}$
- $(DE)^+ = \{ABCDE\}$
- $(ABC)^+ = \{ABCDE\}$

- relation schema $R = \{A, B, C, D, E\}$.
 - ★ $A \rightarrow BC$
 - ★ $CD \rightarrow E$
 - ★ $B \rightarrow D$
 - ★ $E \rightarrow A$

Uses of Attribute Closure

There are several uses of the attribute closure algorithm:

- Testing for superkey:

- ★ To test if α is a superkey, we compute α^+ , and check if α^+ **contains all attributes of R** .

Example of Attribute Set Closure

■ $(AG)^+$

1. $result = AG$

2. $result = ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)

3. $result = ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)

4. $result = ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)

■ Is AG a candidate key?

1. Is AG a super key?

1. Does $AG \rightarrow R$? == Is $(AG)^+ \supseteq R$

2. Is any subset of AG a superkey?

1. Does $A \rightarrow R$? == Is $(A)^+ \supseteq R$

2. Does $G \rightarrow R$? == Is $(G)^+ \supseteq R$

Example of Lossy-Join Decomposition

- Lossy-join decompositions result in information loss.
- Example: Decomposition of $R = (A, B)$

$$R_1 = (A) \quad R_2 = (B)$$

A	B
α	1
α	2
β	1

r

A
α
β

$\Pi_A(r)$

B
1
2

$\Pi_{B(r)}$

$\Pi_A(r) \bowtie \Pi_B(r)$

A	B
α	1
α	2
β	1
β	2

Lossless-join decomposition

Let

R be a relation schema,

F be a set of functional dependencies on **R**.

Let

R1 and **R2** form a decomposition of **R**.

This decomposition is a lossless-join decomposition of **R**

if **at least one of the following functional dependencies is in F^+ :**

$$R_1 \cap R_2 \rightarrow R_1$$

$$R_1 \cap R_2 \rightarrow R_2$$

Decomposition

- Decompose the relation schema *Lending-schema* into:

Branch-schema = (*branch-name*, *branch-city*, *assets*)

Loan-info-schema = (*customer-name*, *loan-number*,
branch-name, *amount*)

- **All attributes** of an original schema (R) must appear in the decomposition (R_1, R_2):

$$R = R_1 \cup R_2$$

- **Lossless-join** decomposition.
For all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is **lossless join** if and only if **at least one of the following dependencies is in F^+** :

- ★ $R_1 \cap R_2 \rightarrow R_1$

- ★ $R_1 \cap R_2 \rightarrow R_2$

Dependency Preservation

$$F = \{A \rightarrow B, B \rightarrow C\}$$

Let decomposed into $\{AC\}$ and $\{AB\}$.

The **restriction** of F to $\{AC\}$ is then $A \rightarrow C$,

Since $A \rightarrow C$ is in F^+

Even though it is not in F .

We have a relation $R(A, B, C)$ with functional dependencies:

$A \rightarrow B$

$B \rightarrow C$

So

logically:

$A \rightarrow B \rightarrow C \Rightarrow A \rightarrow C$

We have a relation $R(A, B, C)$ with functional dependencies:

$A \rightarrow B$

$B \rightarrow C$

So

logically:

$A \rightarrow B \rightarrow C \Rightarrow A \rightarrow C$

A	B	C
a1	b1	c1
a2	b2	c2
a3	b3	c3
a4	b4	c4

A	B
a1	b1
a2	b2
a3	b3
a4	b4

B	C
b1	c1
b2	c2
b3	c3
b4	c4

A	R1.B	R2.B	C
a1	b1	b1	c1
a1	b1	b2	c2
a1	b1	b3	c3
a1	b1	b4	c4
a2	b2	b1	c1
a2	b2	b2	c2
a2	b2	b3	c3
a2	b2	b4	c4
a3	b3	b1	c1
a3	b3	b2	c2
a3	b3	b3	c3
a3	b3	b4	c4
a4	b4	b1	c1
a4	b4	b2	c2
a4	b4	b3	c3
a4	b4	b4	c4

A	R1.B	R2.B	C
a1	b1	b1	c1
a1	b1	b2	c2
a1	b1	b3	c3
a1	b1	b4	c4
a2	b2	b1	c1
a2	b2	b2	c2
a2	b2	b3	c3
a2	b2	b4	c4
a3	b3	b1	c1
a3	b3	b2	c2
a3	b3	b3	c3
a3	b3	b4	c4
a4	b4	b1	c1
a4	b4	b2	c2
a4	b4	b3	c3
a4	b4	b4	c4

A	B	C
a1	b1	c1
a2	b2	c2
a3	b3	c3
a4	b4	c4

A	B	C
a1	b1	c1
a2	b1	c2
a3	b2	c3
a4	b2	c4

A	B
a1	b1
a2	b1
a3	b2
a4	b2

B	C
b1	c1
b1	c2
b2	c3
b2	c4

A	R1.B	R2.B	C
a1	b1	b1	c1
a1	b1	b1	c2
a1	b1	b2	c3 ✖
a1	b1	b2	c4 ✖
a2	b1	b1	c1
a2	b1	b1	c2
a3	b2	b2	c3
a3	b2	b2	c4
a4	b2	b2	c3
a4	b2	b2	c4

$$R1.B = R2.B$$

A	B	C
a1	b1	c1
a1	b1	c2
a2	b1	c1
a2	b1	c2
a3	b2	c3
a3	b2	c4
a4	b2	c3
a4	b2	c4

Functional Dependency Preservation

A	B	C
1	1	1
2	1	2
3	2	1
4	2	2

A	B
1	1
2	1
3	2
4	2

B	C
1	1
1	2
2	1
2	2

$R=\{ABC\}$
 $F= \{BC \rightarrow A, A \rightarrow BC\}$

$R1 = [AB]$
 $F1= \{A \rightarrow B\}$

$R1 = [BC]$
 $F2= \{ \}$

$F' = F1 \cup F2$
 $= \{A \rightarrow B\}$

Therefore, $F' \cong F$ Becomes False
NOT dependency preserving